

Task Questions

1. Question 1

It is port 80. It is responsible for HTTP traffic.

2. Question 2

Yes, if the instances have a public IP address, and the security group and network ACL will allow it

3. Question 3

Yes, the private subnet 1 has the route table configured in a way where all the traffic is routed to the NAT gateway. This will allow the private subnet to connect to the internet

4. Question 4

No, because the private subnet 2's route are not routed to the NAT gateway, it is not configured in such a way, so private subnet will not be able to connect to the internet

5. Question 5

No, even though the web server has a public IPv4 address, the route table hasn't been configured to accept TCP connections. The route table has the configuration enabled for port 80 but not port 22 which is TCP. So, the CafeWebAppServer instance cannot connect from the internet

6. Question 6

It is Café WebServer Image

Module 9 Challenge Lab - Creating a Scalable and Highly Available Environment for the Cafe

Task 1: Inspecting your environment

In this task, I'm tasked with evaluating the current setup of our lab environment. I start by exploring the Amazon VPC console to understand how the network is configured. Then, I move on to answer a series of questions about the lab setup.

View questions in: [English](#)

Question 1: Which ports are open in the *CafeSG* security group?

☐ Ports 80 and 443
☒ Port 80
☐ Ports 80, 443, and 3899
☐ Ports 22, 80, and 443

Question 2: Can you connect from the internet to instances in *Public Subnet 1*?

☒ Yes - If the instance has a public IP address, and the security group and network ACL allow it
☐ No - The public subnet has no internet gateway
☐ No - The public subnet has no NAT gateway configured for it
☐ No - The network access control list (network ACL) prevents any inbound traffic from the internet

Question 3: Should an instance in *Private Subnet 1* be able to reach the internet?

☒ Yes
☐ No

These questions include inquiries about the open ports in the CafeSG security group, the accessibility of instances in Public Subnet 1 from the internet, whether instances in Private Subnets 1 and 2 should have internet access, the ability to connect to the CafeWebAppServer instance from the internet, and finally, identifying the name of the Amazon Machine Image (AMI) being used. After answering these questions, I'll submit my responses to record my findings.

Question 4: Should an instance in *Private Subnet 2* be able to reach the internet?

☐ Yes
☒ No

Question 5: Can you connect to the *CafeWebAppServer* instance from the internet?

☐ Yes
☒ No

Question 6: What is the name of the Amazon Machine Image (AMI)?

☐ Amazon Linux
☐ WebServerAMI
☒ Cafe WebServer Image
☐ My Amazing Image

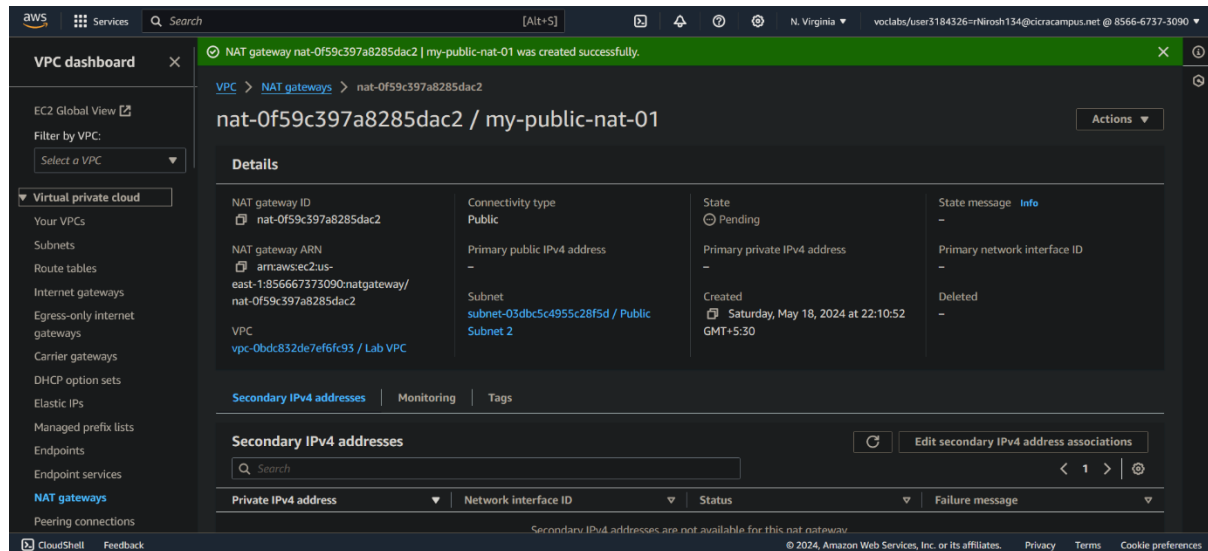
SIT233 – Cloud Computing

Module 9 Challenge Lab - Creating a Scalable and Highly Available Environment for the Cafe

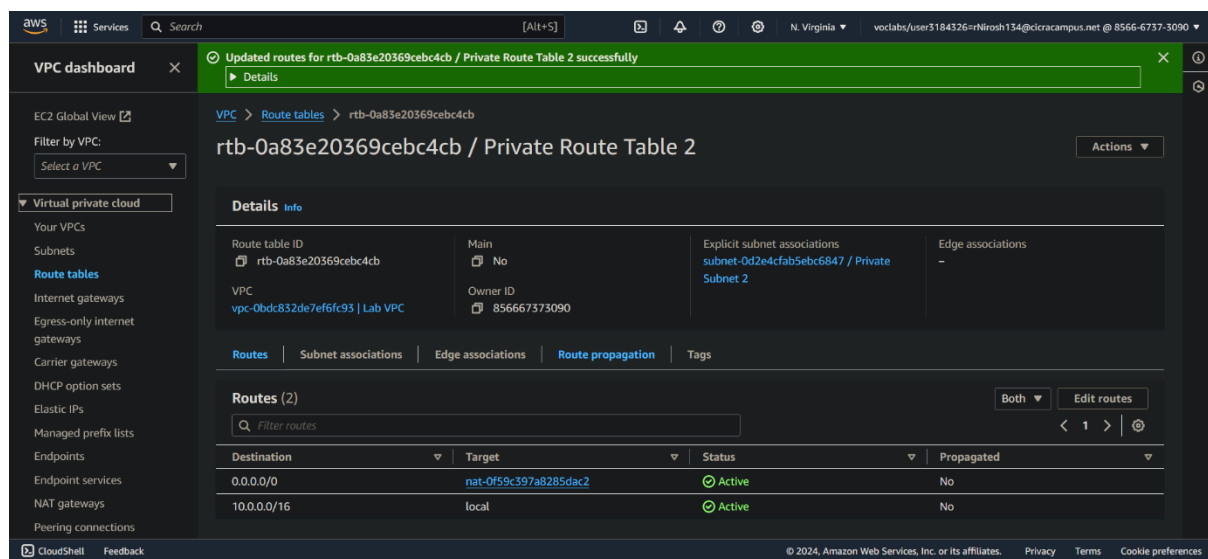
Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Task 2: Creating a NAT gateway for the second Availability Zone

In this task, I'm ensuring high availability for our architecture by spanning it across two Availability Zones. Before launching Amazon EC2 instances for our web application servers in the second Availability Zone, I need to set up a NAT gateway.



This gateway will enable instances without public IP addresses to access the internet. To accomplish this, I create a NAT gateway in the Public Subnet of the second Availability Zone. Then, I configure the network to direct internet-bound traffic from instances in Private Subnet 2 to the newly created NAT gateway.



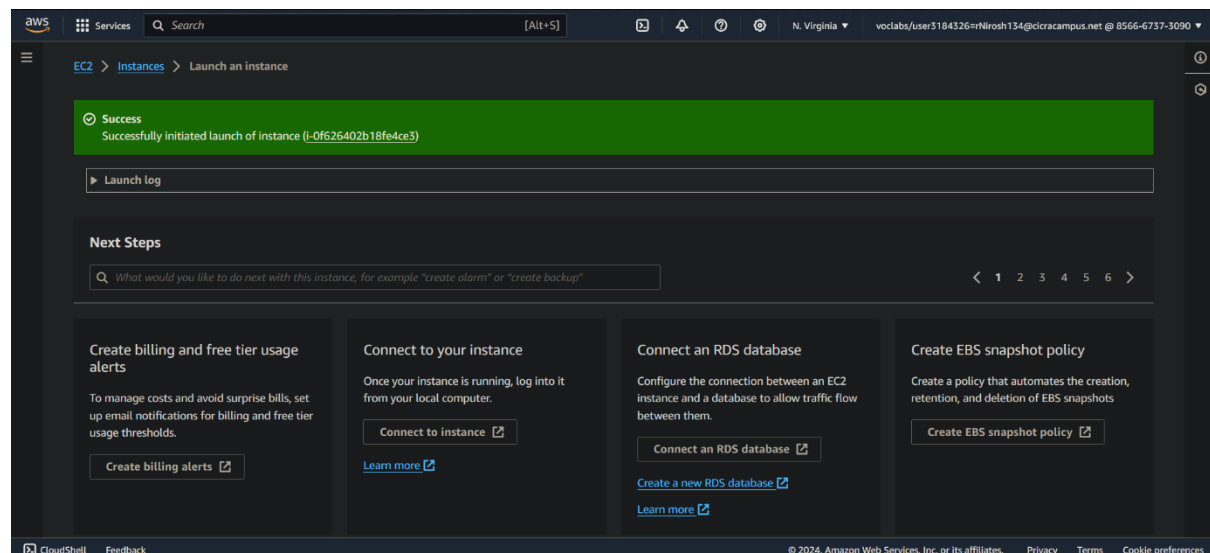
SIT233 – Cloud Computing
Module 9 Challenge Lab - Creating a Scalable and Highly
Available Environment for
the Cafe

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Task 3: Creating a bastion host instance in a public subnet

In this task, I'm setting up a bastion host in a public subnet to facilitate secure access to EC2 instances located in private subnets in later tasks. Using the Amazon EC2 console, I create an EC2 instance in one of the public subnets of the Lab VPC.

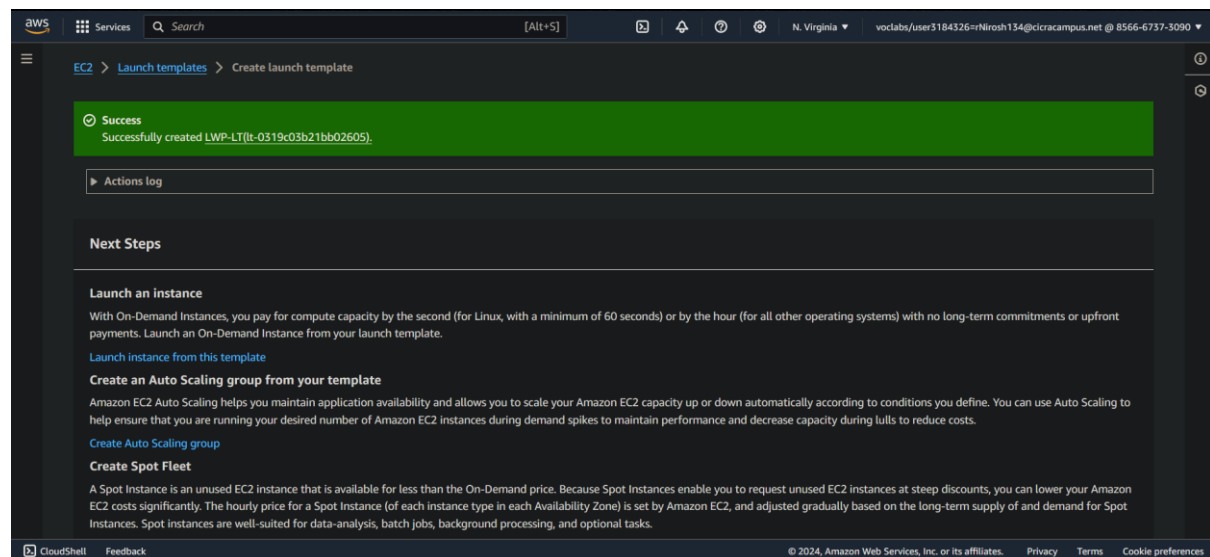
The instance is named "Bastion Host" and uses the Amazon Linux 2023 AMI with a t2.micro instance type. I ensure it uses the "vockey" key pair for authentication and enable the Auto-assign Public IP setting. Additionally, I configure the security group to only allow SSH traffic on port 22 from my IP address, ensuring secure access to the bastion host.



Task 4: Creating a launch template

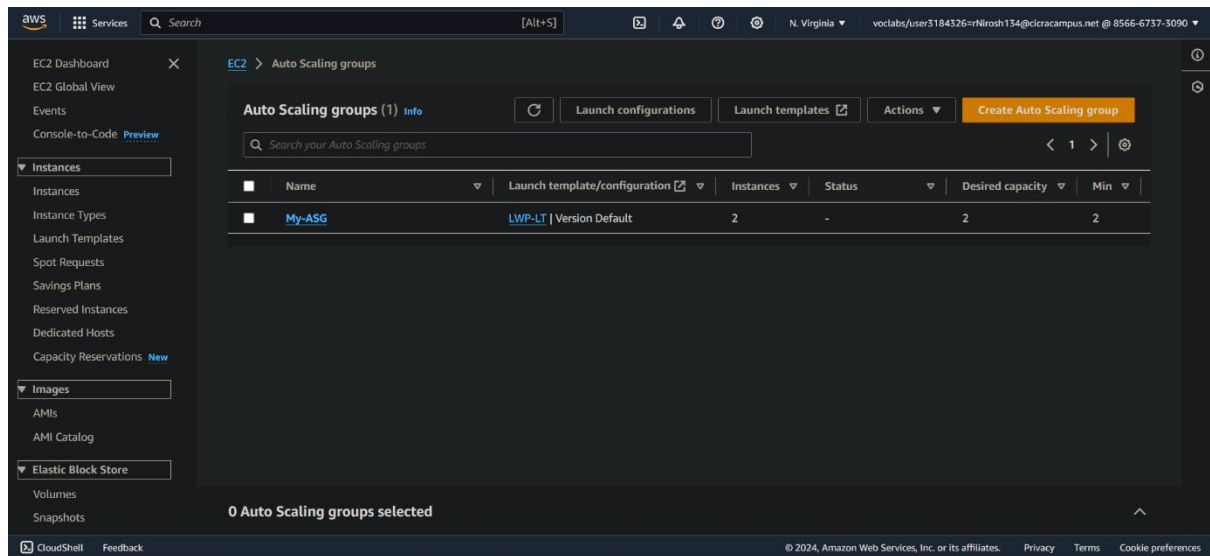
In this task, I'm tasked with creating a launch template using the Amazon Machine Image (AMI) generated from the CafeWebAppServer instance. Using the Amazon EC2 console, I navigate to create a launch template and specify the required configurations. I select the Cafe WebServer Image from the AMI dropdown menu, ensuring I locate the correct image. Then, I choose the t2.micro instance type from the Instance Type dropdown menu.

For security, I create a new key pair for login authentication and select it, ensuring to download the key pair to my local computer. I assign the CafeSG security group to the launch template, ensuring the instance adheres to the required security policies. Additionally, I tag the resource with the key "Name" and value "webserver" and ensure the IAM Instance Profile is set to CafeRole, locating this setting in the Advanced Details section.

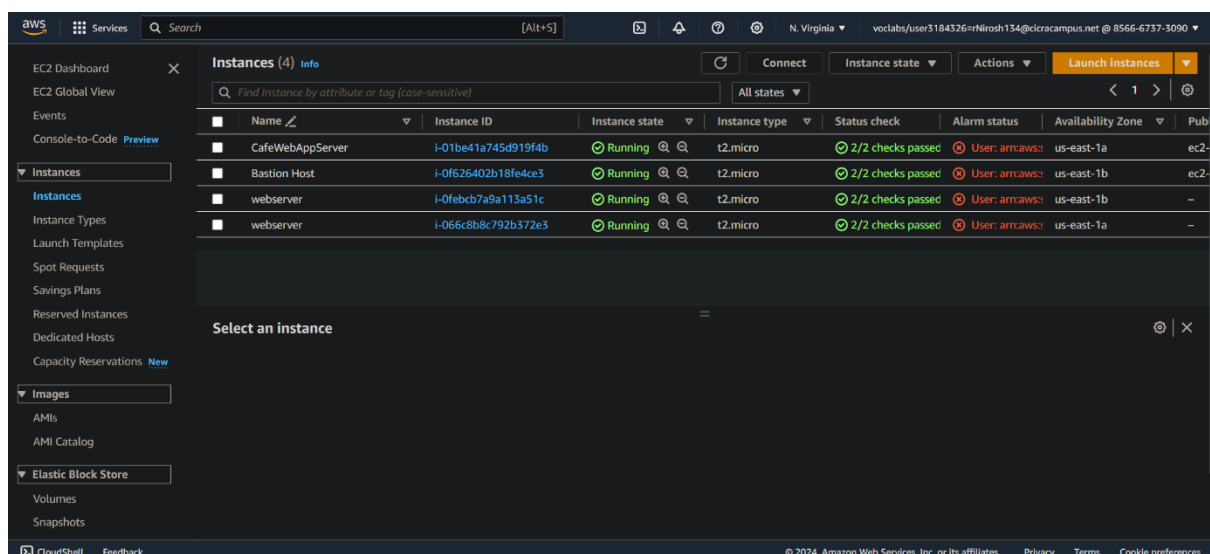


Task 5: Creating an Auto Scaling group

In this task, I'm creating an Auto Scaling group using the launch template defined earlier. I navigate to the Amazon EC2 console and proceed to create the Auto Scaling group without setting up a load balancer at this stage. I ensure that the Auto Scaling group uses the specified launch template and is configured to operate within the VPC designated for the lab. Additionally, I select Private Subnet 1 and Private Subnet 2 for the subnets where the instances will be launched.



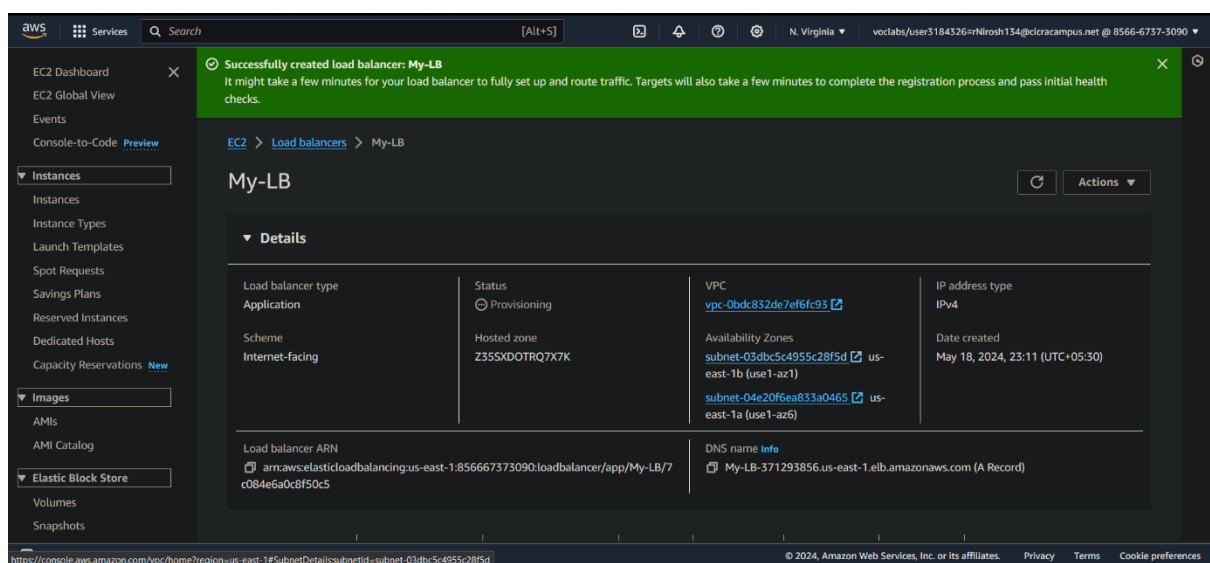
I skip the advanced options and configure the group size parameters: Desired capacity is set to 2, Minimum capacity to 2, and Maximum capacity to 6. Furthermore, I enable the Target tracking scaling policy with the metric type set to Average CPU utilization and a target value of 25, requiring instances within 60 units. After creating the Auto Scaling group, I verify its configuration in the Amazon EC2 console, ensuring that two instances are present, both tagged with the specified resource tags from the previous task.



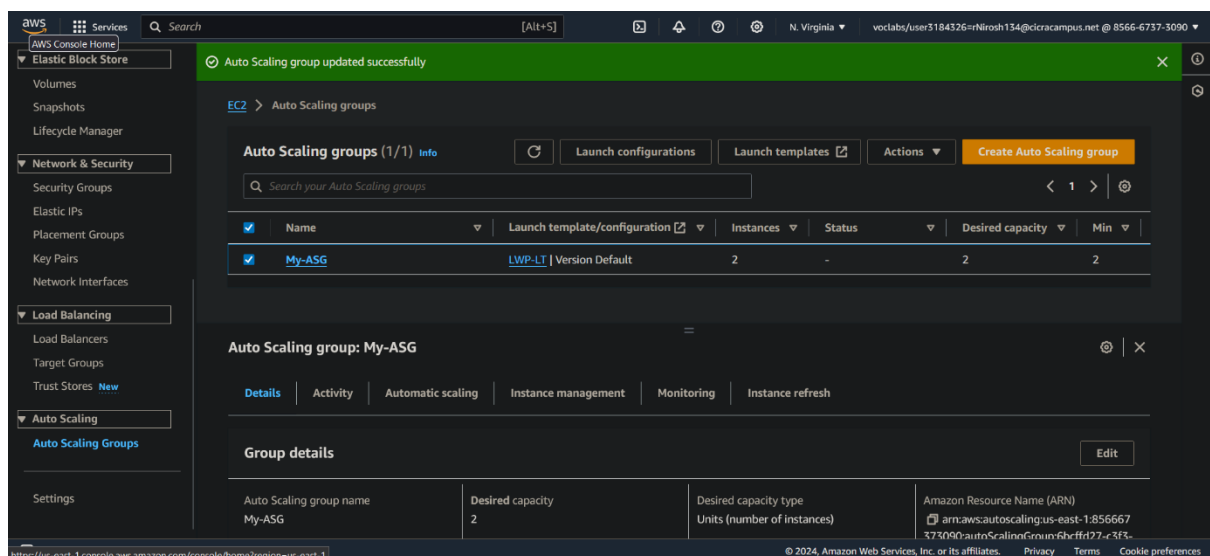
Task 6: Creating a load balancer

In this task, I'm setting up a load balancer to allow external connections to my web application server instances deployed in private subnets. I create an HTTP Application Load Balancer within the VPC configured for the lab, utilizing the two public subnets. I skip the HTTPS security settings and create a new security group to allow HTTP traffic from anywhere.

Additionally, I create a new target group for the load balancer but skip registering targets for now. After creating the load balancer, I modify the Auto Scaling group created earlier to include this new load balancer, adding the target group in the configuration.



Once the load balancer is active, I'll proceed to test the café's web application to ensure it functions properly and scales automatically under load, completing the process of updating the application architecture.

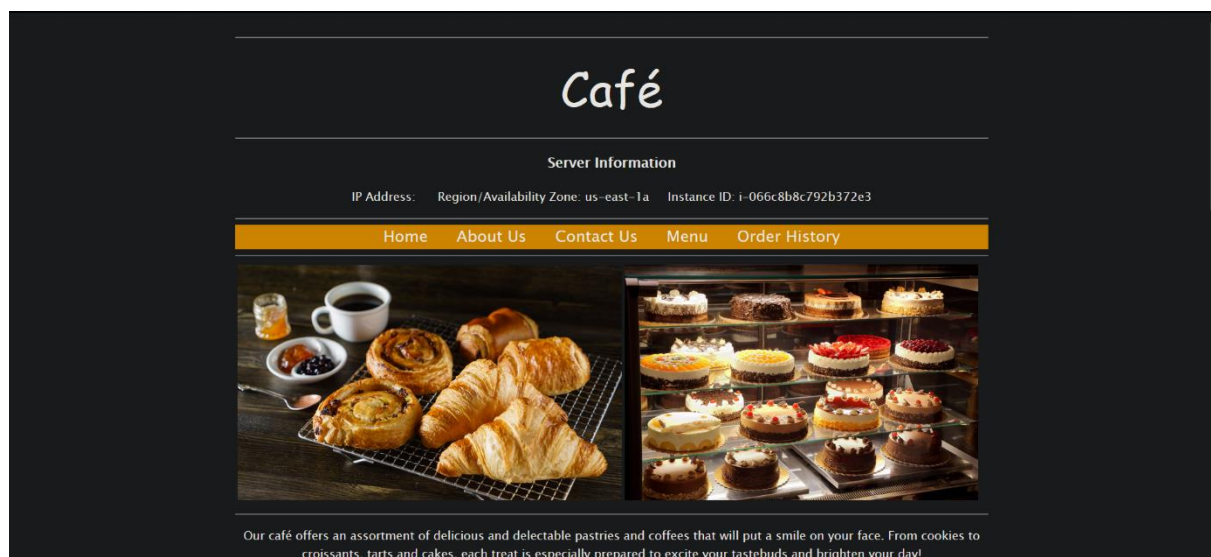


Task 7: Testing the web application

In this task, I'll test the café web application by visiting the Domain Name System (DNS) name of the load balancer and appending /café to the URL. If the café application loads successfully, everything is set up correctly. However, if it doesn't load, I need to review my work in the lab tasks.

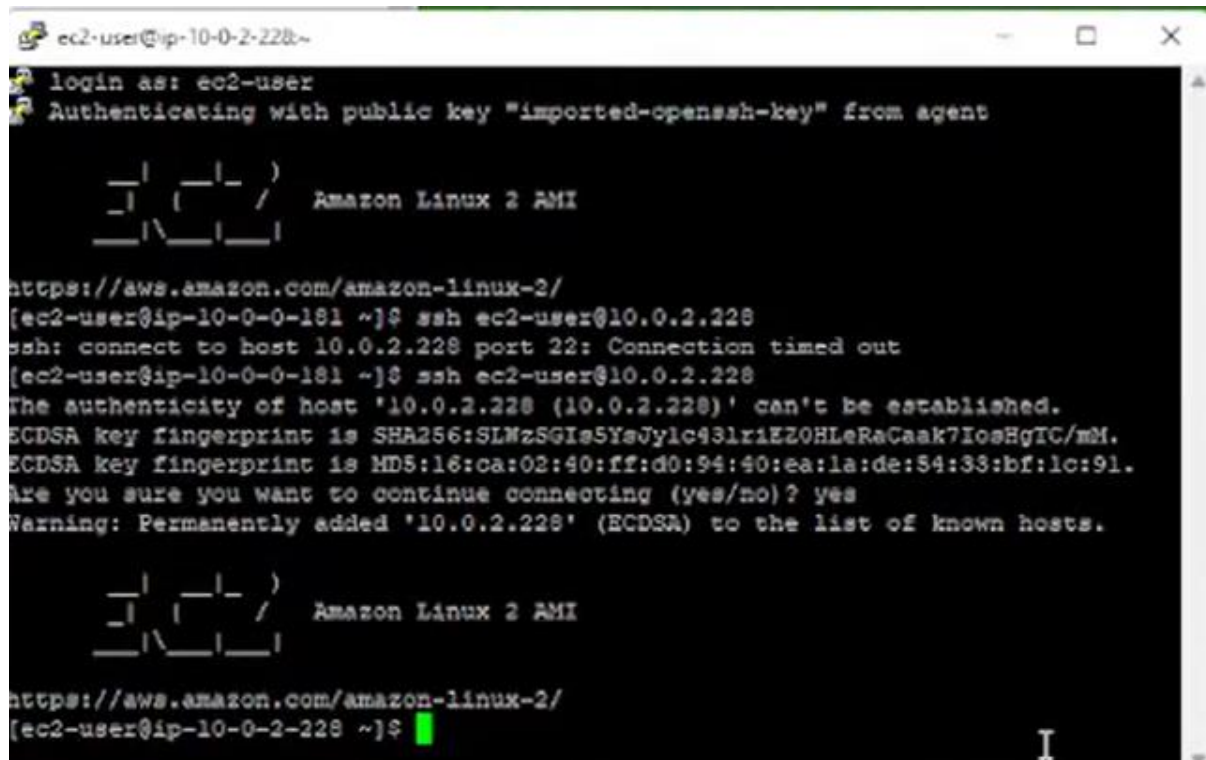
I'll check if the NAT gateway was added correctly, if the route tables were updated with the NAT gateway, if the instance specifies an IAM role, if the load balancer is in the public subnets, if the instances were deployed from the Auto Scaling group in the correct subnets, and if the security groups allow HTTP traffic from the internet. T

his ensures that all components are properly configured for the café web application to function correctly.



Task 8: Testing automatic scaling under load

In this task, I'll test whether the café application can automatically scale out under load. First, I'll use Secure Shell (SSH) passthrough through the bastion host instance to connect to one of the running web server instances. I'll need to modify the CafeSG security group to allow SSH traffic over port 22 from the bastion host. Then, from the web server instance, I'll run a stress test using specific commands to increase the load on the web server CPU.

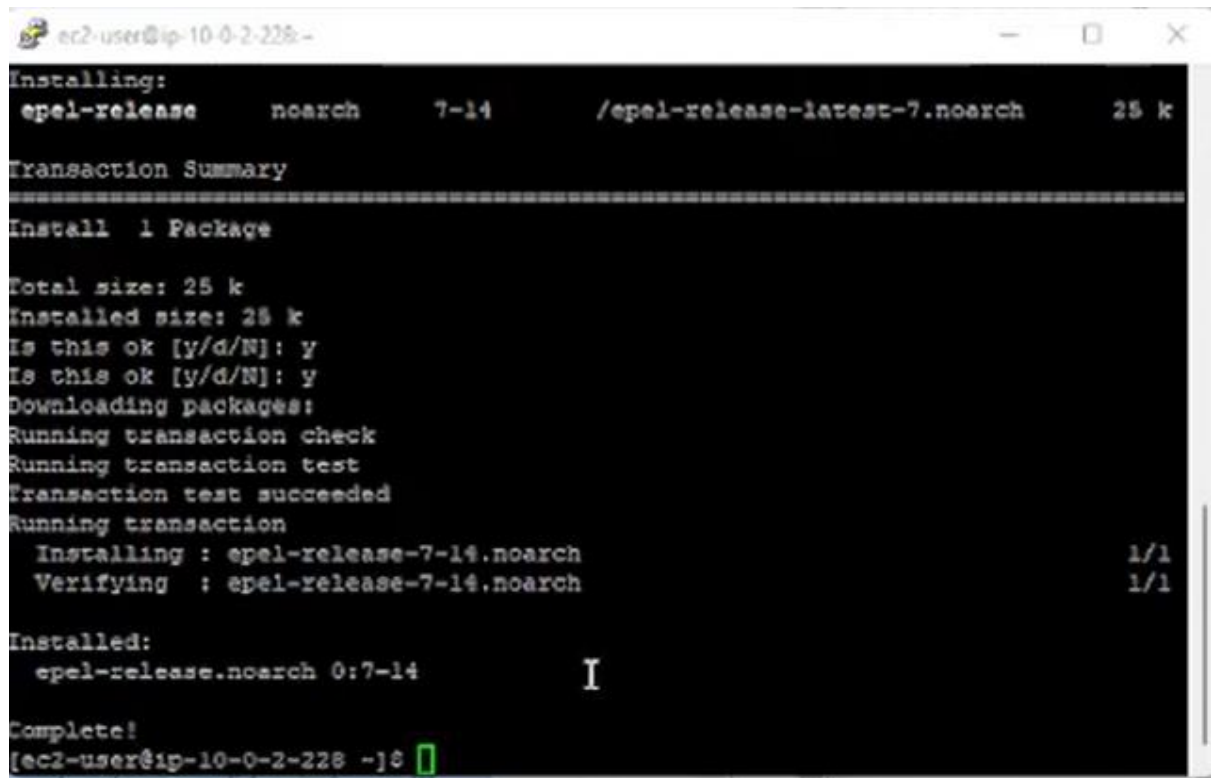


```
ec2-user@ip-10-0-2-228~  
login as: ec2-user  
Authenticating with public key "imported-openssh-key" from agent  
  
  _| _| _|  
 _| ( _| _| / Amazon Linux 2 AMI  
 _| \ _| _|  
  
https://aws.amazon.com/amazon-linux-2/  
[ec2-user@ip-10-0-0-181 ~]$ ssh ec2-user@10.0.2.228  
ssh: connect to host 10.0.2.228 port 22: Connection timed out  
[ec2-user@ip-10-0-0-181 ~]$ ssh ec2-user@10.0.2.228  
The authenticity of host '10.0.2.228 (10.0.2.228)' can't be established.  
ECDSA key fingerprint is SHA256:SLWzSGIe5YeJylc43lr1EZ0HLeRaCaak7IosHgIC/mM.  
ECDSA key fingerprint is MD5:16:ca:02:40:ff:d0:94:40:ea:1a:de:54:33:bf:1c:91.  
Are you sure you want to continue connecting (yes/no)? yes  
Warning: Permanently added '10.0.2.228' (ECDSA) to the list of known hosts.  
  
  _| _| _|  
 _| ( _| _| / Amazon Linux 2 AMI  
 _| \ _| _|  
  
https://aws.amazon.com/amazon-linux-2/  
[ec2-user@ip-10-0-2-228 ~]$
```

SIT233 – Cloud Computing
Module 9 Challenge Lab - Creating a Scalable and Highly
Available Environment for
the Cafe

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

While the stress test is running, I'll monitor the Amazon EC2 console to observe if the Auto Scaling group deploys new instances to handle the increased load. This will help me determine if the café application can effectively scale out automatically when faced with higher demand.



```
ec2-user@ip-10-0-2-228:~$ sudo yum install https://dl.fedoraproject.org/pub/epel/epel-release-latest-7.noarch.rpm
Installing:
 epel-release      noarch      7-14      /epel-release-latest-7.noarch      25 k

Transaction Summary
=====
Install 1 Package

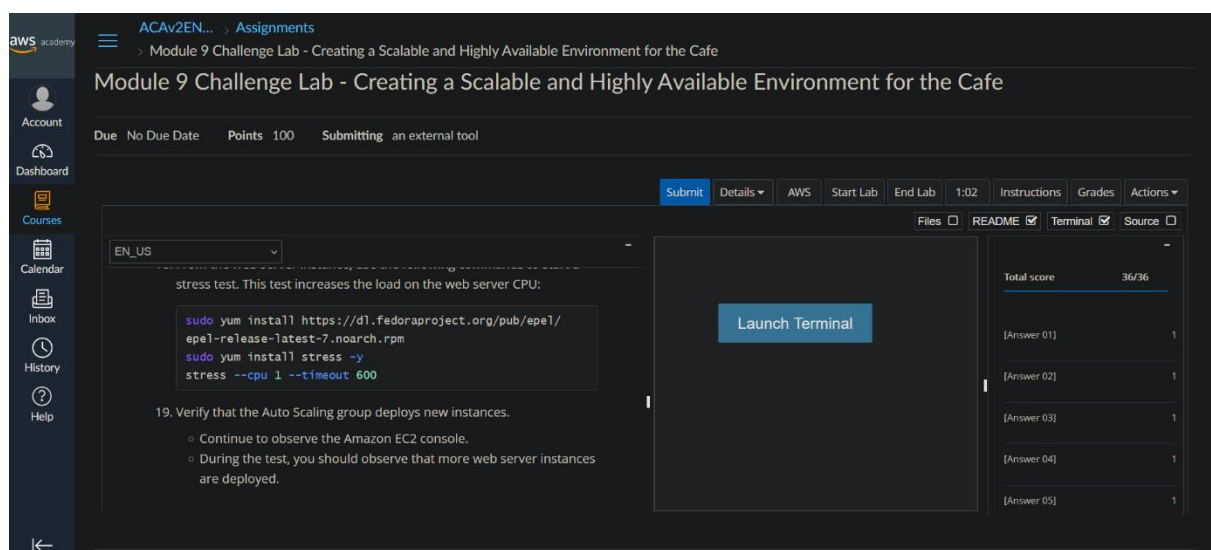
Total size: 25 k
Installed size: 25 k
Is this ok [y/d/N]: y
Is this ok [y/d/N]: y
Downloading packages:
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : epel-release-7-14.noarch      1/1
  Verifying  : epel-release-7-14.noarch      1/1

Installed:
 epel-release.noarch 0:7-14

Complete!
[ec2-user@ip-10-0-2-228 ~]$
```

Grade

My grade is 36/36



The screenshot shows the AWS Academy interface for a challenge lab. The lab title is "Module 9 Challenge Lab - Creating a Scalable and Highly Available Environment for the Cafe". It has a due date of "No Due Date", is worth "Points 100", and is "Submitting an external tool". The lab progress bar shows "100%". The lab instructions are visible, including a terminal window with the following commands:

```
sudo yum install https://dl.fedoraproject.org/pub/epel/epel-release-latest-7.noarch.rpm
sudo yum install stress -y
stress --cpu 1 --timeout 600
```

The lab instructions also mention verifying that the Auto Scaling group deploys new instances during the stress test. On the right side, the "Grades" section shows a "Total score" of "36/36" and five individual question scores, each marked as "1".